Práctica N°5: Comunicación UART

Índice

1. Objetivos **2**
- 1.1. Objetivo General . . . 2
- 1.2. Objetivos Específicos . . . 2

2. Marco Teórico **2**
- 2.1. Comunicación UART . . . 2
- 2.2. Parámetros de configuración . . . 3
- 2.3. Uso de PuTTY para la comunicación . . . 4

3. Descripción del Código **6**
- 3.1. Módulo `UART_RX` . . . 6
- 3.2. Módulo `UART_TX` . . . 9
- 3.3. Módulo `UART_ECHO` . . . 12
- 3.4. Módulo `uart_echo_top` . . . 18

4. Asignación de Pines (XDC) **20**
- 4.1. Mapa de pines (top-level) . . . 20
- 4.2. Parámetros eléctricos (IOSTANDARD) . . . 20
- 4.3. Archivo `.xdc` (fuente) . . . 21

5. Resultados **21**

1. Objetivos

1.1. Objetivo General

Establecer la comunicación UART entre el ordenador y la tarjeta Nexys A7-100T, permitiendo la transmisión y recepción de caracteres a través de un software de terminal (como PuTTY), y comprendiendo de manera práctica el funcionamiento de este protocolo de comunicación serie.

1.2. Objetivos Específicos

- Configurar la interfaz UART en la tarjeta Nexys para recibir y enviar datos.
- Utilizar un software de terminal (PuTTY) para establecer conexión con la tarjeta, enviar caracteres y observar las respuestas.
- Implementar la lógica en VHDL para que la tarjeta:
  - Reenvíe (echo) los caracteres numéricos recibidos.
  - Genere el mensaje `'error'` cuando se reciban caracteres no numéricos.
- Comprender el flujo completo de comunicación UART desde la configuración del hardware hasta la interacción con el software en el ordenador.
- Promover el uso de herramientas de simulación y prueba que faciliten la depuración y comprensión del sistema.

2. Marco Teórico

2.1. Comunicación UART

La comunicación **UART** (Universal Asynchronous Receiver/Transmitter) es un protocolo de transmisión serie asíncrono ampliamente utilizado para el intercambio de datos entre dispositivos electrónicos. Su característica principal es que no requiere un reloj compartido entre emisor y receptor, ya que ambos extremos deben estar configurados con la misma velocidad de transmisión (*baud rate*).

En este protocolo los datos se transmiten bit a bit a través de una línea, siguiendo un formato estándar:

- **Bit de inicio (Start bit):** indica al receptor que comienza la transmisión.
- **Bits de datos:** normalmente 7 u 8 bits, que representan la información a transmitir.
- **Bit de paridad (opcional):** se utiliza para la detección de errores simples.
- **Bits de parada (Stop bits):** uno o dos bits que señalan el final de la transmisión.

En la Figura 1 se muestra el formato de una trama UART típica.

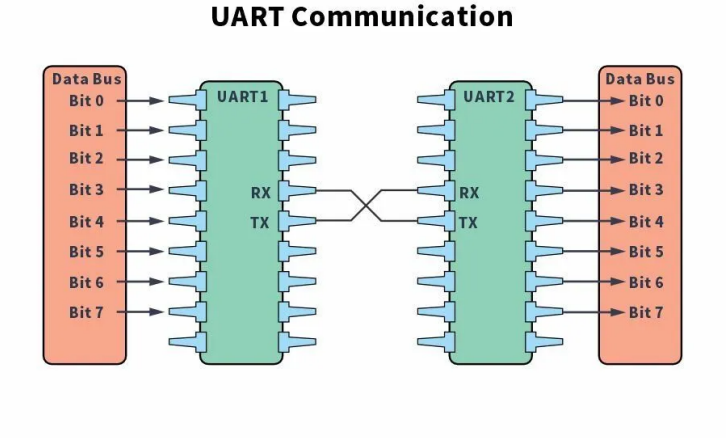


Figura 1: Estructura básica de una trama UART.

2.2. Parámetros de configuración

Para que la comunicación UART funcione correctamente, tanto el ordenador como la tarjeta Nexys deben compartir los mismos parámetros:

- **Velocidad de transmisión (baud rate):** número de bits enviados por segundo (ejemplo: 115200 bps).
- **Bits de datos:** cantidad de bits que representan cada carácter (8 bits en la mayoría de los casos).
- **Paridad:** puede ser `None`, `Even` o `Odd`. En esta práctica no se usará paridad.
- **Bits de parada:** normalmente se usa un bit de parada.

2.3. Uso de PuTTY para la comunicación

PuTTY es un software de terminal que permite establecer comunicación serie con la tarjeta Nexys mediante el puerto COM del ordenador. Su uso facilita la prueba e interacción con el diseño implementado en VHDL.

Los pasos principales para su configuración son los siguientes:

1. Instalar PuTTY en el ordenador (descarga gratuita desde https://www.putty.org/).
2. Conectar la tarjeta Nexys al PC mediante el cable USB.
3. Identificar el puerto COM asignado a la tarjeta en el administrador de dispositivos.
4. Abrir PuTTY y seleccionar la opción **Serial** en la pantalla inicial.
5. Configurar el puerto COM detectado, la velocidad (9600), 8 bits de datos, sin paridad, y 1 bit de parada (formato 8N1).

En las siguientes Figuras se muestra la ventana de configuración de PuTTY con los parámetros requeridos.

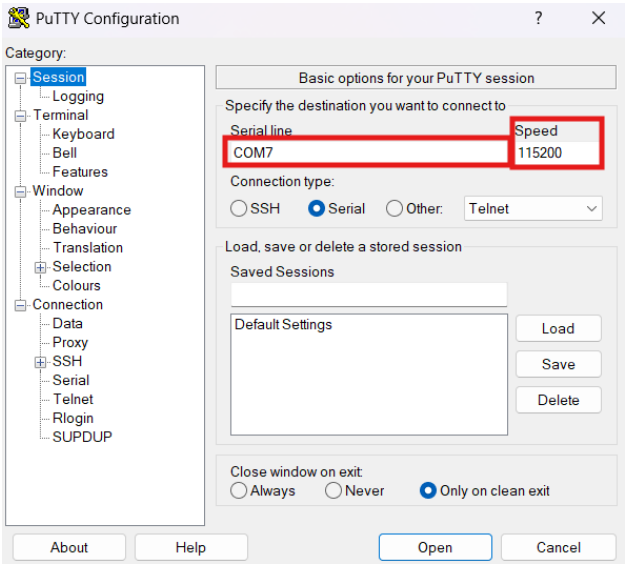


Figura 2: Configuración de PuTTY para la práctica de comunicación UART referente al puerto COM y baud rate.

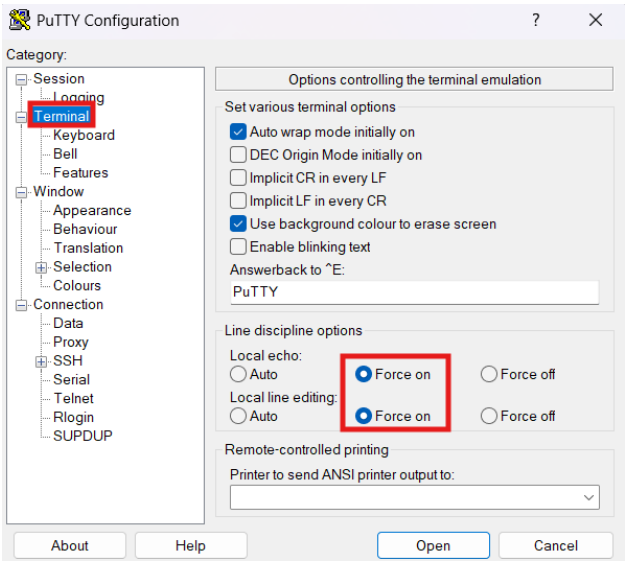


Figura 3: Configuración de PuTTY para la práctica de comunicación UART referente a la habilitación de envió.

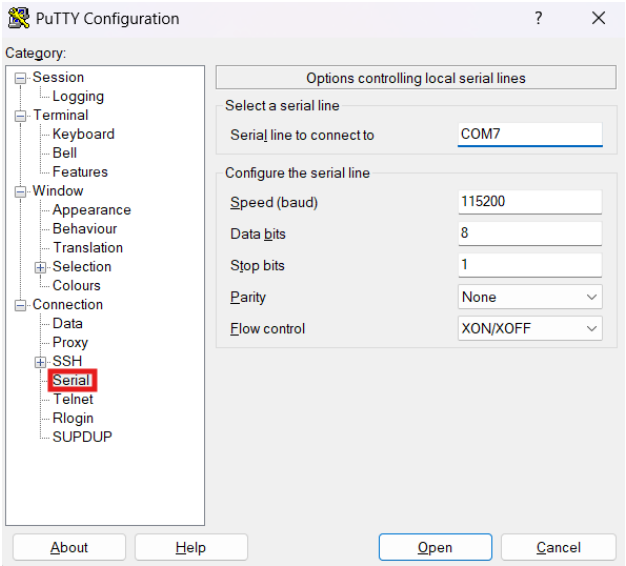


Figura 4: Configuración de PuTTY para la práctica de comunicación UART referente al serial.

3. Descripción del Código

3.1. Módulo UART_RX

El módulo `uart_rx` implementa el receptor UART. Su función es detectar los bits que llegan por la línea serie `rx`, reconstruir el byte transmitido y notificar mediante la señal `rx_done` cuando un dato válido ha sido recibido.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity uart_rx is
6      Port (
7          clk        : in  std_logic;
8          rst        : in  std_logic;
9          rx         : in  std_logic;
10         data_out   : out std_logic_vector(7 downto 0);
11         rx_done    : out std_logic
12     );
13 end uart_rx;
14
15 architecture Behavioral of uart_rx is
16
17     constant CLK_FREQ     : integer := 100_000_000; --
           100 MHz
18     constant BAUD_RATE    : integer := 115200;
19     constant TICKS_PER_BIT: integer := CLK_FREQ /
           BAUD_RATE;  -- ~868
20
21     type state_type is (IDLE, START, DATA, STOP);
22     signal state      : state_type := IDLE;
23
24     signal bit_count  : integer range 0 to 7 := 0;
25     signal tick_count : integer range 0 to TICKS_PER_BIT
           - 1 := 0;
26     signal shift_reg  : std_logic_vector(7 downto 0) :=
           (others => '0');
27     signal rx_sync    : std_logic_vector(1 downto 0) :=
           (others => '1');
28
29 begin
30     -- Sincronizaci n de la entrada
31     process(clk)
```

```
32      begin
33          if rising_edge(clk) then
34              rx_sync(0) <= rx;
35              rx_sync(1) <= rx_sync(0);
36          end if;
37      end process;
38
39      -- M quina de estados del receptor
40      process(clk)
41      begin
42          if rising_edge(clk) then
43              if rst = '1' then
44                  state <= IDLE;
45                  bit_count <= 0;
46                  tick_count <= 0;
47                  shift_reg <= (others => '0');
48                  rx_done <= '0';
49              else
50                  rx_done <= '0';  -- valor por defecto
51                  case state is
52                      when IDLE =>
53                          if rx_sync(1) = '0' then  --
                                detectar bit de inicio
54                              tick_count <= TICKS_PER_BIT
                                    / 2;
55                              state <= START;
56                          end if;
57
58                      when START =>
59                          if tick_count = 0 then
60                              if rx_sync(1) = '0' then
61                                  tick_count <=
                                        TICKS_PER_BIT - 1;
62                                  bit_count <= 0;
63                                  state <= DATA;
64                              else
65                                  state <= IDLE; -- falso
                                        inicio
66                              end if;
67                          else
68                              tick_count <= tick_count -
                                    1;
69                          end if;
```

```
70
71                      when DATA =>
72                          if tick_count = 0 then
73                              shift_reg(bit_count) <=
                                   rx_sync(1);
74                              if bit_count = 7 then
75                                  state <= STOP;
76                              else
77                                  bit_count <= bit_count +
                                       1;
78                              end if;
79                              tick_count <= TICKS_PER_BIT
                                   - 1;
80                          else
81                              tick_count <= tick_count -
                                   1;
82                          end if;
83
84                      when STOP =>
85                          if tick_count = 0 then
86                              if rx_sync(1) = '1' then
87                                  data_out <= shift_reg;
88                                  rx_done <= '1';
89                              end if;
90                              state <= IDLE;
91                          else
92                              tick_count <= tick_count -
                                   1;
93                          end if;
94
95                      when others =>
96                          state <= IDLE;
97                  end case;
98              end if;
99          end if;
100     end process;
101 end Behavioral;
```

Listing 1: Código completo del módulo UART_RX

Explicación del módulo:

- **Constantes:** se define la frecuencia del reloj de la FPGA (100 MHz), la velocidad de transmisión (115200 bps) y los ciclos por bit (`TICKS_PER_BIT`).
- **Estados:** la FSM tiene cuatro estados: `IDLE`, `START`, `DATA` y `STOP`.
- **Sincronización:** la señal `rx` pasa por dos flip-flops (`rx_sync`) para evitar problemas de metastabilidad.
- **Proceso principal:**
  - En `IDLE` se espera a que la línea baje (bit de inicio).
  - En `START` se valida el inicio y se prepara el conteo de bits.
  - En `DATA` se van muestreando los 8 bits y guardando en `shift_reg`.
  - En `STOP` se valida el bit de parada y se activa `rx_done`.
- **Salida:** el dato recibido se entrega en `data_out` y la señal `rx_done` avisa que está listo.

—

3.2. Módulo `UART_TX`

El módulo `uart_tx` implementa el transmisor UART. Toma un byte de entrada y lo envía bit a bit por la línea `tx` con el formato estándar: bit de inicio, 8 bits de datos y bit de parada.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity uart_tx is
6      Port (
7          clk      : in  std_logic;
8          rst      : in  std_logic;
9          tx_start : in  std_logic;
10         data_in  : in  std_logic_vector(7 downto 0);
11         tx       : out std_logic;
12         tx_busy  : out std_logic
13     );
14 end uart_tx;
15
16 architecture Behavioral of uart_tx is
```

```
17
18      constant CLK_FREQ     : integer := 100_000_000; --
            100 MHz
19      constant BAUD_RATE    : integer := 115200;
20      constant TICKS_PER_BIT: integer := CLK_FREQ /
            BAUD_RATE;  -- ~868
21
22      type state_type is (IDLE, START, DATA, STOP);
23      signal state      : state_type := IDLE;
24
25      signal shift_reg  : std_logic_vector(7 downto 0) :=
            (others => '0');
26      signal bit_count  : integer range 0 to 7 := 0;
27      signal tick_count : integer range 0 to TICKS_PER_BIT
            - 1 := 0;
28      signal tx_reg     : std_logic := '1';
29
30  begin
31      tx <= tx_reg;
32      tx_busy <= '1' when state /= IDLE else '0';
33
34      process(clk)
35      begin
36          if rising_edge(clk) then
37              if rst = '1' then
38                  state <= IDLE;
39                  tick_count <= 0;
40                  bit_count <= 0;
41                  tx_reg <= '1';
42              else
43                  case state is
44                      when IDLE =>
45                          tx_reg <= '1';
46                          if tx_start = '1' then
47                              shift_reg <= data_in;
48                              tick_count <= TICKS_PER_BIT
                                    - 1;
49                              state <= START;
50                          end if;
51
52                      when START =>
53                          tx_reg <= '0';
54                          if tick_count = 0 then
```

```
55                              tick_count <= TICKS_PER_BIT
                                   - 1;
56                              bit_count <= 0;
57                              state <= DATA;
58                          else
59                              tick_count <= tick_count -
                                   1;
60                          end if;
61
62                      when DATA =>
63                          tx_reg <= shift_reg(bit_count);
64                          if tick_count = 0 then
65                              if bit_count = 7 then
66                                  state <= STOP;
67                              else
68                                  bit_count <= bit_count +
                                       1;
69                              end if;
70                              tick_count <= TICKS_PER_BIT
                                   - 1;
71                          else
72                              tick_count <= tick_count -
                                   1;
73                          end if;
74
75                      when STOP =>
76                          tx_reg <= '1';
77                          if tick_count = 0 then
78                              state <= IDLE;
79                          else
80                              tick_count <= tick_count -
                                   1;
81                          end if;
82
83                      when others =>
84                          state <= IDLE;
85                  end case;
86              end if;
87          end if;
88      end process;
89
90  end Behavioral;
```

Listing 2: Código completo del módulo UART_TX

Explicación del módulo:

- **Entradas:** el reloj, el reset y la señal `tx_start`, que indica cuándo transmitir.
- **Datos:** el byte a transmitir entra en `data_in` y se almacena en un registro de desplazamiento.
- **FSM:** similar a `UART_RX`, con estados `IDLE`, `START`, `DATA` y `STOP`.
- **Señal `tx_busy`:** se activa cuando hay una transmisión en curso, útil para evitar solapamientos.
- **Secuencia:**
  1. En `IDLE`, la línea está en nivel alto.
  2. En `START`, se envía un `0`.
  3. En `DATA`, se envían los 8 bits del byte.
  4. En `STOP`, la línea vuelve a nivel alto y termina la transmisión.

3.3. Módulo `UART_ECHO`

El módulo `uart_echo` orquesta la lógica de eco/validación: acumula los caracteres recibidos por `UART_RX` en un buffer; cuando llega `Enter` (CR, `x"0D"`), decide si todos los caracteres fueron dígitos y, según el caso, transmite el eco (más `CR+LF`) o el mensaje fijo *'error'* mediante `UART_TX`.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity uart_echo is
6      Port (
7          clk : in std_logic;
8          rst : in std_logic;
9          rx  : in std_logic;
10         tx  : out std_logic
11     );
12 end uart_echo;
13
14 architecture Behavioral of uart_echo is
15
16     -- Componentes UART
```

```
17      component uart_rx
18          Port (
19              clk      : in  std_logic;
20              rst      : in  std_logic;
21              rx       : in  std_logic;
22              data_out : out std_logic_vector(7 downto 0);
23              rx_done  : out std_logic
24          );
25      end component;
26
27      component uart_tx
28          Port (
29              clk      : in  std_logic;
30              rst      : in  std_logic;
31              tx_start : in  std_logic;
32              data_in  : in  std_logic_vector(7 downto 0);
33              tx       : out std_logic;
34              tx_busy  : out std_logic
35          );
36      end component;
37
38      -- Par metros y tipos
39      constant BUFFER_LEN : integer := 16;
40      type buffer_type is array(0 to BUFFER_LEN - 1) of
           std_logic_vector(7 downto 0);
41
42      -- Se ales internas
43      signal rx_data     : std_logic_vector(7 downto 0);
44      signal rx_done     : std_logic;
45      signal rx_done_d   : std_logic := '0';
46      signal rx_pulse    : std_logic := '0';
47
48      signal tx_data     : std_logic_vector(7 downto 0);
49      signal tx_start    : std_logic := '0';
50      signal tx_busy     : std_logic;
51
52      signal input_buffer : buffer_type := (others => (
           others => '0'));
53      signal input_index  : integer range 0 to BUFFER_LEN
           := 0;
54      signal valid_input  : boolean := true;
55      signal send_index   : integer range 0 to BUFFER_LEN
           := 0;
```

```
56
57      type state_type is (
58          IDLE, ACCUMULATE, CHECK_ENTER,
59          SEND_DATA, SEND_CR, SEND_LF,
60          SEND_ERROR, SEND_ERR_LOOP,
61          RESET_BUFFER
62      );
63      signal state : state_type := IDLE;
64
65      constant error_msg : buffer_type := (
66          x"65", x"72", x"72", x"6F", x"72", x"0D", x"0A",
                -- "error\r\n"
67          others => x"00"
68      );
69      signal error_len : integer := 7;
70      signal err_index : integer range 0 to 7 := 0;
71
72  begin
73
74      -- Instancias
75      uart_rx_inst : uart_rx port map(clk, rst, rx,
           rx_data, rx_done);
76      uart_tx_inst : uart_tx port map(clk, rst, tx_start,
           tx_data, tx, tx_busy);
77
78      -- Flanco de subida de rx_done
79      process(clk)
80      begin
81          if rising_edge(clk) then
82              rx_done_d <= rx_done;
83              rx_pulse  <= rx_done and not rx_done_d;
84          end if;
85      end process;
86
87      -- FSM principal
88      process(clk)
89      begin
90          if rising_edge(clk) then
91              if rst = '1' then
92                  state <= IDLE;
93                  input_index <= 0;
94                  send_index <= 0;
95                  err_index <= 0;
```

```
96                  valid_input <= true;
97                  tx_start <= '0';
98              else
99                  tx_start <= '0';  -- default
100
101                 case state is
102                     when IDLE =>
103                         if rx_pulse = '1' then
104                             if rx_data = x"0D" then  -- CR
105                                 state <= CHECK_ENTER;
106                             else
107                                 input_buffer(input_index) <= rx_data;
108                                 if input_index < BUFFER_LEN - 1 then
109                                     input_index <= input_index + 1;
110                                 end if;
111                                 -- Verifica si es un digito
112                                 if rx_data < x"30" or rx_data > x"39" then
113                                     valid_input <= false;
114                                 end if;
115                                 state <= ACCUMULATE;
116                             end if;
117                         end if;
118
119                     when ACCUMULATE =>
120                         state <= IDLE;
121
122                     when CHECK_ENTER =>
123                         if valid_input = true and input_index > 0 then
124                             send_index <= 0;
125                             state <= SEND_DATA;
126                         else
127                             err_index <= 0;
128                             state <= SEND_ERROR;
129                         end if;
130
```

```
131                         when SEND_DATA =>
132                             if tx_busy = '0' then
133                                 tx_data <= input_buffer(
                                        send_index);
134                                 tx_start <= '1';
135                                 state <= SEND_CR;
136                             end if;
137
138                         when SEND_CR =>
139                             if tx_busy = '0' then
140                                 if send_index < input_index
                                        - 1 then
141                                     send_index <= send_index
                                            + 1;
142                                     state <= SEND_DATA;
143                                 else
144                                     tx_data <= x"0D"; -- CR
145                                     tx_start <= '1';
146                                     state <= SEND_LF;
147                                 end if;
148                             end if;
149
150                         when SEND_LF =>
151                             if tx_busy = '0' then
152                                 tx_data <= x"0A"; -- LF
153                                 tx_start <= '1';
154                                 state <= RESET_BUFFER;
155                             end if;
156
157                         when SEND_ERROR =>
158                             if tx_busy = '0' then
159                                 tx_data <= error_msg(
                                        err_index);
160                                 tx_start <= '1';
161                                 state <= SEND_ERR_LOOP;
162                             end if;
163
164                         when SEND_ERR_LOOP =>
165                             if tx_busy = '0' then
166                                 if err_index < error_len - 1
                                        then
167                                     err_index <= err_index +
                                            1;
```

```
168                                     state <= SEND_ERROR;
169                                 else
170                                     state <= RESET_BUFFER;
171                                 end if;
172                             end if;
173
174                         when RESET_BUFFER =>
175                             input_index <= 0;
176                             valid_input <= true;
177                             state <= IDLE;
178
179                         when others =>
180                             state <= IDLE;
181                     end case;
182                 end if;
183             end if;
184     end process;
185
186 end Behavioral;
```

Listing 3: Código completo del módulo UART_ECHO

Cómo funciona y por qué está escrito así

- **Acumulación de entrada:** cada byte recibido (`rx_pulse='1'`) se guarda en `input_buffer(input_index)`. El índice aumenta hasta `BUFFER_LEN-1`. Si llegan más caracteres, se dejan de indexar (pero `valid_input` puede seguir cambiando).
- **Criterio de validez:** se marca `valid_input=false` si algún byte está fuera de `x"30"`.. `x"39"` (ASCII de '0' a '9').
- **Disparo por Enter:** cuando el byte es `x"0D"` (CR), se evalúa: si `valid_input=true` y hay al menos un carácter, se envía el eco (`SEND_DATA` → `SEND_CR` → `SEND_LF`); si no, se transmite `'error'` (`SEND_ERROR` → `SEND_ERR_LOOP`).
- **Secuencia de TX:** `tx_start='1'` inicia cada envío cuando `tx_busy='0'`. Tras cada byte, la FSM espera a que `tx_busy` libere para programar el siguiente.
- **Limpieza:** `RESET_BUFFER` pone el sistema listo para una nueva línea: `input_index=0` y `valid_input=true`.

3.4. Módulo `uart_echo_top`

Este módulo es el *top level* de la práctica: expone los puertos físicos de la placa (reloj, reset y líneas UART) y encapsula la lógica de eco/validación implementada en `uart_echo`. Su responsabilidad es puramente estructural: instanciar y cablear el bloque funcional.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity uart_echo_top is
    Port (
        clk     : in  std_logic;  -- 100 MHz clock
        rst     : in  std_logic;  -- reset s ncrono
           activo en alto
        rx      : in  std_logic;  -- UART RX (desde PC)
        tx      : out std_logic   -- UART TX (hacia PC)
    );
end uart_echo_top;

architecture Structural of uart_echo_top is

    component uart_echo
        Port (
            clk : in std_logic;
            rst : in std_logic;
            rx  : in std_logic;
            tx  : out std_logic
        );
    end component;

begin

    uart_inst : uart_echo
        port map (
            clk => clk,
            rst => rst,
            rx  => rx,
            tx  => tx
        );

end Structural;
```

Listing 4: Código completo del módulo UART_ECHO_TOP

Puertos y función

- `clk`: reloj de sistema a 100 MHz de la Nexys A7. Debe corresponderse con la frecuencia usada por `UART_RX` y `UART_TX` (100 MHz) para que el cálculo de `TICKS_PER_BIT` sea coherente.
- `rst`: *reset* **síncrono**, activo en alto. Se muestrea en flanco de subida del reloj dentro de `uart_echo`.
- `rx`: línea de recepción desde el puente USB–UART de la placa (PC → FPGA).
- `tx`: línea de transmisión hacia el puente USB–UART (FPGA → PC).

Estructura e integración

- **Instanciación única**: el `component uart_echo` se instancia como `uart_inst`; no hay lógica adicional en el `top`, favoreciendo la separación "*plumbing*" vs "*behavior*".
- **Acoplamiento con el hardware**: mapea `rx`/`tx` a los pines conectados al puente USB–UART de la Nexys A7 (FTDI). En la guía, indica al estudiante verificar el manual para los pines del canal UART de la placa y configurar `IOSTANDARD LVCMOS33`.
- **Reset y botón físico**: como el reset es *síncrono*, si lo conectas a un pulsador físico, su *rebote* no rompe la sincronía, pero sí puede causar múltiples resets breves. Didácticamente, sugiere interponer un *debouncer* sencillo si el alumno observa comportamiento errático.

Pruebas rápidas (en laboratorio)

1. Sintetiza y descarga el `bitstream`. Abre PuTTY en modo **Serial** con 115200–8N1 (como en la Sección de PuTTY).
2. Teclea una secuencia de dígitos (p. ej., `123`) y presiona `Enter`: debes recibir `123` seguido de `CR+LF`.
3. Teclea algo con letras (p. ej., `12a` + `Enter`): debes recibir `error` (o `Error` si adoptaste esa variante) seguido de `CR+LF`.

4. Asignación de Pines (XDC)

En esta práctica, el tope `uart_echo_top` expone cuatro puertos: `clk`, `rst`, `rx` y `tx`. La siguiente tabla resume el mapeo físico (paquete) según el archivo `.xdc`.

4.1. Mapa de pines (top-level)

Cuadro 1: Correspondencia entre puertos lógicos y pines físicos del FPGA

Señal (VHDL)	**Dirección**	**PACKAGE_PIN**	**Función**
`clk`	Entrada	E3	Reloj 100 MHz
`rst`	Entrada	M17	Reset síncrono
`rx`	Entrada	C4	UART RX (PC→FPGA)
`tx`	Salida	D4	UART TX (FPGA→PC)

4.2. Parámetros eléctricos (IOSTANDARD)

Todos los pines se configuran a `LVCMOS33`, compatible con la IO de la Nexys A7 y el puente USB–UART de la placa.

Cuadro 2: Estandar eléctrico aplicado a cada puerto

Señal	**PACKAGE_PIN**	**IOSTANDARD**
`clk`	E3	LVCMOS33
`rst`	M17	LVCMOS33
`rx`	C4	LVCMOS33
`tx`	D4	LVCMOS33

4.3. Archivo `.xdc` (fuente)

Para trazabilidad, el `.xdc` utilizado es:

```
set_property PACKAGE_PIN E3  [get_ports clk]
set_property PACKAGE_PIN M17 [get_ports rst]
set_property PACKAGE_PIN C4  [get_ports rx]
set_property PACKAGE_PIN D4  [get_ports tx]

set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rx]
set_property IOSTANDARD LVCMOS33 [get_ports tx]
```

Listing 5: Fragmento XDC utilizado

5. Resultados

En esta sección se valida el comportamiento completo del sistema `uart_echo_top` usando **PuTTY** como terminal serie. Se realizó una única sesión en modo **Serial** configurada a **115200–8N1** (sin paridad, 1 bit de parada), donde se probaron dos casos:

1. **Entrada numérica válida**: envío de una secuencia de dígitos, p. ej., `123`, seguida de `Enter`. La placa debe responder con el *eco* de la secuencia y finalizar con `CR+LF`.

2. **Entrada no numérica**: envío de una secuencia que contenga al menos un carácter no numérico, p. ej., `12a`, seguida de `Enter`. La placa debe responder con el mensaje `Error` (o `error`, según lo definido en el módulo `UART_ECHO`) y finalizar con `CR+LF`.



Figura 5: Evidencia en PuTTY: eco de un numero y respuesta de `Error/error` para caracteres no numericos, ambos terminados en `CR+LF`.